```python
import random
import math
import numpy as np
import matplotlib.pyplot as plt


GRID_SIZE = 10
START_CELL = (0, 0)
TARGET_CELL = (9, 9)

OBSTACLES = {
    (3, 2), (3, 3), (3, 4), (3, 5),
    (6, 5), (6, 6), (6, 7), (7, 5)
}

class RobotGridACO:
    def __init__(self, num_ants=25, iterations=40, alpha=1.0, beta=2.0, evaporation_rate=0.2, Q=10):
        self.num_ants = num_ants
        self.iterations = iterations
        self.alpha = alpha
        self.beta = beta
        self.rho = evaporation_rate
        self.Q = Q
        self.max_steps = 50

        self.pheromones = np.ones((GRID_SIZE, GRID_SIZE))

    def _get_heuristic(self, cell):
        """Calculates attractiveness based on straight-line distance to target."""
        dist = math.sqrt((cell[0] - TARGET_CELL[0])**2 + (cell[1] - TARGET_CELL[1])**2)
        return 1.0 / (dist + 0.1)

    def _get_valid_neighbors(self, current_cell, visited_cells):
        """Finds neighboring grid squares that are within boundaries and open."""
        neighbors = []
        directions = [(-1,0), (1,0), (0,-1), (0,1), (-1,-1), (-1,1), (1,-1), (1,1)]

        for dx. dv in directions:
```

```
        for dx, dy in directions:
            nx, ny = current_cell[0] + dx, current_cell[1] + dy
            neighbor = (nx, ny)

            if 0 <= nx < GRID_SIZE and 0 <= ny < GRID_SIZE:
                if neighbor not in OBSTACLES and neighbor not in visited_cells:
                    neighbors.append(neighbor)
        return neighbors

    def _simulate_single_ant(self):
        """Simulates one ant trying to navigate from Start to Target."""
        current = START_CELL
        path = [current]
        visited = {current}

        for _ in range(self.max_steps):
            if current == TARGET_CELL:
                return path

            neighbors = self._get_valid_neighbors(current, visited)
            if not neighbors:
                break

            probabilities = []
            for n in neighbors:
                tau = self.pheromones[n[0]][n[1]] ** self.alpha
                eta = self._get_heuristic(n) ** self.beta
                probabilities.append(tau * eta)

            prob_sum = sum(probabilities)
            if prob_sum == 0:
                probabilities = [1.0 / len(neighbors)] * len(neighbors)
            else:
                probabilities = [p / prob_sum for p in probabilities]

            next_cell = random.choices(neighbors, weights=probabilities, k=1)[0]
            path.append(next_cell)
            visited.add(next_cell)
```

```
            path.append(next_cell)
            visited.add(next_cell)
            current = next_cell

        return None

    def _calculate_path_length(self, path):
        """Calculates the physical geometric distance of the path."""
        distance = 0.0
        for i in range(len(path) - 1):
            p1, p2 = path[i], path[i+1]
            distance += math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
        return distance

    def optimize(self):
        best_path = None
        best_distance = float('inf')

        for iteration in range(self.iterations):
            successful_paths = []
            path_distances = []

            for _ in range(self.num_ants):
                path = self._simulate_single_ant()
                if path:
                    dist = self._calculate_path_length(path)
                    successful_paths.append(path)
                    path_distances.append(dist)

                    if dist < best_distance:
                        best_distance = dist
                        best_path = path

            self.pheromones *= (1 - self.rho)

            for path, dist in zip(successful_paths, path_distances):
                deposit = self.Q / dist
                for cell in path:
                    self.pheromones[cell[0]][cell[1]] += deposit
```

```
        return best_path, best_distance

def generate_2d_path_plot(path):
    """
    Renders a 2D map showing the grid layout, solid obstacle blocks,
    and the final optimized trajectory line.
    """
    plt.figure(figsize=(8, 8))

    plt.grid(True, which='both', color='gainsboro', linestyle='-', linewidth=1)

    for (ox, oy) in OBSTACLES:
        plt.gca().add_patch(plt.Rectangle((ox - 0.5, oy - 0.5), 1.0, 1.0,
                                          facecolor='tomato', edgecolor='darkred', alpha=0.8))

    plt.plot([], [], color='tomato', label='Obstacle Block', marker='s', markersize=10, linestyle='')

    path_x = [cell[0] for cell in path]
    path_y = [cell[1] for cell in path]

    plt.plot(path_x, path_y, color='dodgerblue', linewidth=3, marker='o', markersize=6, label='Robot Path Trajectory')
    plt.scatter(START_CELL[0], START_CELL[1], color='lime', s=180, edgecolor='black', zorder=5, label='Start Point')
    plt.scatter(TARGET_CELL[0], TARGET_CELL[1], color='gold', s=180, edgecolor='black', zorder=5, label='Target Destination')

    plt.title("2D Spatial Path Map (Ant Colony Optimization)", fontsize=14, pad=15)
    plt.xlabel("Grid X Coordinates", fontsize=11)
    plt.ylabel("Grid Y Coordinates", fontsize=11)

    plt.xlim(-0.5, GRID_SIZE - 0.5)
    plt.ylim(-0.5, GRID_SIZE - 0.5)
    plt.xticks(range(GRID_SIZE))
    plt.yticks(range(GRID_SIZE))

    plt.legend(loc='upper left', shadow=True)
    plt.gca().set_aspect('equal', adjustable='box')
    plt.show()

if __name__ == "__main__":
```

```
    for (ox, oy) in OBSTACLES:
        plt.gca().add_patch(plt.Rectangle((ox - 0.5, oy - 0.5), 1.0, 1.0,
                                          facecolor='tomato', edgecolor='darkred', alpha=0.8))

    plt.plot([], [], color='tomato', label='Obstacle Block', marker='s', markersize=10, linestyle='')

    path_x = [cell[0] for cell in path]
    path_y = [cell[1] for cell in path]

    plt.plot(path_x, path_y, color='dodgerblue', linewidth=3, marker='o', markersize=6, label='Robot Path Trajectory')
    plt.scatter(START_CELL[0], START_CELL[1], color='lime', s=180, edgecolor='black', zorder=5, label='Start Point')
    plt.scatter(TARGET_CELL[0], TARGET_CELL[1], color='gold', s=180, edgecolor='black', zorder=5, label='Target Destination')

    plt.title("2D Spatial Path Map (Ant Colony Optimization)", fontsize=14, pad=15)
    plt.xlabel("Grid X Coordinates", fontsize=11)
    plt.ylabel("Grid Y Coordinates", fontsize=11)

    plt.xlim(-0.5, GRID_SIZE - 0.5)
    plt.ylim(-0.5, GRID_SIZE - 0.5)
    plt.xticks(range(GRID_SIZE))
    plt.yticks(range(GRID_SIZE))

    plt.legend(loc='upper left', shadow=True)
    plt.gca().set_aspect('equal', adjustable='box')
    plt.show()

if __name__ == "__main__":
    aco_planner = RobotGridACO(num_ants=25, iterations=45, alpha=1.0, beta=2.0)
    optimal_path, minimum_distance = aco_planner.optimize()

    if optimal_path:
        print(f"Success! Found a safe route with a length of {minimum_distance:.2f} units.")
        print(f"Total movement steps taken: {len(optimal_path) - 1}")

        generate_2d_path_plot(optimal_path)
    else:
        print("Ants were unable to find a clear path. Try lowering obstacle densities.")


Success! Found a safe route with a length of 14.49 units.
Total movement steps taken: 12
```

```
Success! Found a safe route with a length of 14.49 units.
Total movement steps taken: 12
```

